

# Type Dependency and Expectation Graph

how utu finds every type error in one pass, and explains each one

Every AST node gets two vertices, its **type** and its **type expectation**, and most nodes use only one — many use neither. A single directed edge set connects them: edges originate at type vertices, carry a transform function, and point in the direction of blame, so the head's type is a consequence of the tail's. An edge landing on a **type** vertex propagates a type forward; an edge landing on an **expectation** vertex records what its context requires. Edges bind; they never fail. Checking runs in two phases: flood types along every edge to a fixpoint, comparing nothing; then walk every node once and check its type against its expectation. Because no comparison happens until the flood settles, one pass finds every error in the program instead of the first failure hiding the rest — and because both endpoints are reified, explaining a failure is just walking in-edges back from each vertex to its seed.

□ type vertex    □ expectation vertex    □ mismatch

## Two vertices per node

The typechecker never asks “what is the type of this expression?” in isolation. It asks two separate questions and stores both answers:



Figure 1: The single invariant. Everything else exists to make this check total and explainable.

Most nodes populate exactly one vertex. Many populate neither — an `if` keyword, a delimiter, a block that is never used as a value. **An unpopulated vertex is the normal case, not an error state.** A node with no type is not a node with a wrong type, and that distinction is what keeps a single unresolved name from poisoning everything downstream of it.

## Edges

There is one directed edge set. Edges always **originate** at a type vertex; they **terminate** at either kind. Direction is blame direction — following an edge backwards answers “why does this node have this type?” Every edge carries a transform  $\tau$  applied as the type crosses it.

An edge is a **binding**, not an assertion: it says where the value in the head vertex came from. Edges cannot fail. Types flow out of annotations and into everything downstream, intermediate values included, and that flow is all an edge represents. The only thing that fails is the phase-2 comparison at a node, and that comparison is not an edge.

**No edge connects a node's own type to its own expectation.** That pairing is precisely what the typechecker checks, and drawing it would confuse binding with checking. An expression's type is never derived from its own context — with one deliberate exception, literals, below.

## Propagation: type $\rightarrow$ type

Moves a known type forward through a structural relationship.

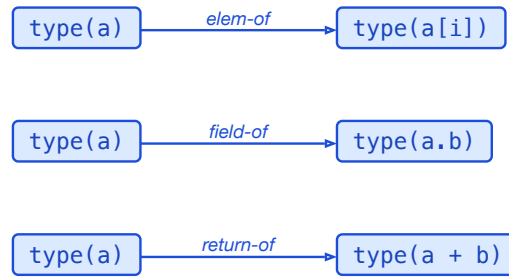


Figure 2: Propagation edges. The transform is what makes `a[i]` narrower than `a`.

## Binding an expectation: `type` → `expectation`

Populates **another** node's expectation from a declared type. Still a binding — it records what the context requires; the requirement is tested later.

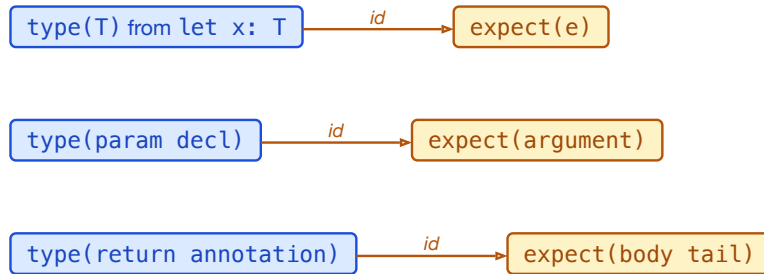


Figure 3: Expectation bindings. Declared types are the seeds; everything downstream is checked against them.

## Expectation vertices are sinks

No edge originates at an expectation vertex. Expectations do not flow onward and never feed a type vertex — **there is no backward pass**.

## The one exception: literals

Literals are the single place where context legitimately **determines** a type rather than constraining it. `let x: I64 = 0` must not fail — the literal has no inherent width, and `std/LiteralDefaults.utu` supplies a type only for when nothing better is known. So the declared context binds the literal's type directly: an edge into `type(literal)`, not `expect(literal)`.

Which types a literal may adopt is declared by the `stdlib` and matched **by name**:

```
<ir-default kind="int" type-name="I32" adopts="I32 U32 I64 U64 M32 M64"/>
```

`Bool` is a wasm `i32` exactly like `I32` is, and is deliberately absent, so `takes_bool(1)` stays an error. Keying adoption on representation instead of name would silently accept it — the same class of bug `TYPES.md` exists to prevent.

## Worked example: `c: t3 = a[b]`

Given `a: t1[t2]`, the graph for `c: t3 = a[b]` is two independent chains that meet at one node:

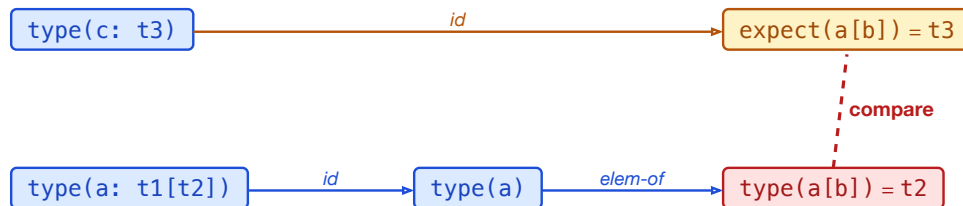


Figure 4: Neither chain is computed at the moment of failure; both already exist.

If `t2` and `t3` are incompatible, blame is a backwards walk from each vertex to its seed:

a[b] is t2 because a was declared t1[t2] at line X;  
it must be t3 because c was declared t3 at line Y.

Both halves, mechanically, **without any diagnostic site having authored either**. This is the property ad-hoc checking structurally cannot have: a checker that computes the expectation at the point of failure has already discarded where that expectation came from.

## Worked example: a.b = c

Assignment chains both edge kinds — three propagation steps and one expectation:



Figure 5: Dot access links type to type; = links type to expectation.

The expectation lands on exactly the node where an error would be reported, and the propagation chain leading into it is the explanation.

## Confluence

When two sources would reach one vertex, they are **chained in source order** rather than joined. The first occurrence becomes the authority; every later one has its expectation bound from it.

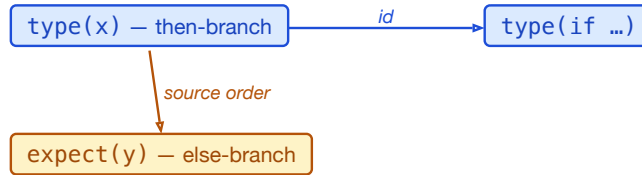


Figure 6: if c { x } else { y }. The first branch types the expression; the second is checked against it.

This is deliberate. A lattice join would need a least-upper-bound over a nominal type system that does not have one, and would erase which branch came first — losing exactly the blame direction that makes the error explainable.

## Checking is two phases



Figure 7: Nothing is checked until propagation has settled.

Deferring all comparison until the flood settles is what makes whole-program error reporting possible: nothing aborts early, so no failure hides the errors behind it.

It is also what makes the editor experience fault-tolerant. A node that is untypeable produces **no comparison** rather than a wrong one, so a single unresolved name does not cascade into its enclosing block, its function, and every caller. Its expectation is still populated from context, so hover still reports expected I32 even when the value is broken.

## The graph is built during the flood, not before it

a + b carries a return-of transform — but **which** overload, and therefore which  $\tau$ , is not known until type(a) settles. Edge creation interleaves with propagation. The implementation is a worklist, not a static graph followed by a traversal.

## Coercion sites fall out for free

The nodes where type(n) and expect(n) are incompatible-but-coercible are exactly the insertion points for an implicit conversion — already enumerated by phase 2, already addressable. utu keeps the set deliberately small:

- $T \rightarrow ?T$  — nullable widening
- $\text{variant} \rightarrow \text{enum}$
- $\text{fun}(\dots) R \rightarrow \text{cl}(\dots) R$  — closure decay

**Coercion never selects an overload.** Resolve the overload from the receiver's actual type first, then coerce arguments to the resolved signature. `utu` has overloaded operators but not overload sets — exactly one candidate per (type, operation) — and this rule is what keeps operator resolution a map lookup rather than a ranked search over conversion sequences.

## Relationship to the nominal/representation split

Propagation is over `type-name` only. The `repr-of` transform is the single edge kind that crosses into `type-repr`, and it appears only on edges consumed by codegen. A transform that reads `type-repr` on a checking edge violates the invariant in `TYPES.md` and does not land.